

Tasneem Mazhar, 2527389, tasneem.mazhar.252@northsouth.edu

Problem 1.

Consider the electrostatic potential $u(r)$ between two concentric spheres of radii $R_1 = 2$ and $R_2 = 4$. The potential of the inner sphere is kept constant at $V_1 = 110$ volts, and the potential of the outer sphere is 0 volts. The potential in the region between the two spheres satisfies

$$\frac{1}{r^2} \frac{d}{dr} \left[r^2 \frac{du}{dr} \right] = 0, \quad R_1 \leq r \leq R_2, \quad u(R_1) = 110, \quad u(R_2) = 0.$$

1. Approximate $u(r)$ at $r = 2.5, 3.0, 3.5$ using
 - (i) the Linear Shooting Method
 - (ii) the Finite Difference Method.
2. Compare the results with the exact solution

$$u(r) = \frac{110 R_1}{r} \left(\frac{R_2 - r}{R_2 - R_1} \right).$$

3. Write your comments on the strength of the methods based on your numerical observations.

Answer :

```

1
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 a = 2
6 b = 4
7 alph = 110
8 beta = 0
9 n = 4
10 m = 3
11
12 # =====
13 # Part 1 (a-i): Linear Shooting Method
14 # =====
15 u = {}
16 v = {}
17 h = (b - a) / n
18
19
20 u[1,0] = alph
21 u[2,0] = 0
22 v[1,0] = 0
23 v[2,0] = 1
24
25 for i in range(n):
26     x = a + i*h
27

```

```

28     k11 = h*u[2,i]
29     k12 = h*((-2/x)*u[2,i])
30     k21 = h*(u[2,i] + 0.5*k12)
31     k22 = h*((-2/(x+h/2))*(u[2,i]+0.5*k12))
32     k31 = h*(u[2,i] + 0.5*k22)
33     k32 = h*((-2/(x+h/2))*(u[2,i]+0.5*k22))
34     k41 = h*(u[2,i] + k32)
35     k42 = h*((-2/(x+h))*(u[2,i]+k32))
36     u[1,i+1] = u[1,i] + (1/6)*(k11+2*k21+2*k31+k41)
37     u[2,i+1] = u[2,i] + (1/6)*(k12+2*k22+2*k32+k42)
38
39     m11 = h*v[2,i]
40     m12 = h*((-2/x)*v[2,i])
41     m21 = h*(v[2,i]+0.5*m12)
42     m22 = h*((-2/(x+h/2))*(v[2,i]+0.5*m12))
43     m31 = h*(v[2,i]+0.5*m22)
44     m32 = h*((-2/(x+h/2))*(v[2,i]+0.5*m22))
45     m41 = h*(v[2,i]+m32)
46     m42 = h*((-2/(x+h))*(v[2,i]+m32))
47     v[1,i+1] = v[1,i] + (1/6)*(m11+2*m21+2*m31+m41)
48     v[2,i+1] = v[2,i] + (1/6)*(m12+2*m22+2*m32+m42)
49
50
51     w = {}
52     w[1,0] = alph
53     w[2,0] = (beta - u[1,n]) / v[1,n]
54
55     # =====
56     # Part 1 (a-ii): Finite Difference Method
57     # =====
58     h1 = (b - a) / (m + 1)
59     a_1 = np.zeros(m+1)
60     b_1 = np.zeros(m+1)
61     c = np.zeros(m+1)
62     d = np.zeros(m+1)
63     l = np.zeros(m+1)
64     u1 = np.zeros(m+1)
65     z = np.zeros(m+1)
66     w_1 = np.zeros(m+2)
67
68     x1 = a + h1
69     a_1[1] = 2
70     b_1[1] = -1+(h1/2)*(-2/x1)
71     d[1] = (1+(h1/2)*(-2/x1))*alph
72
73     for i in range(2, m):
74         x1 = a + i*h1
75         a_1[i] = 2
76         b_1[i] = -1+(h1/2)*(-2/x1)
77         c[i] = -1-(h1/2)*(-2/x1)
78         d[i] = 0
79
80     x1 = b - h1
81     a_1[m] = 2
82     c[m] = -1-(h1/2)*(-2/x1)
83     d[m] = (1-(h1/2)*(-2/x1))*beta
84
85     l[1] = a_1[1]

```

```

86 u1[1] = b_1[1]/a_1[1]
87 z[1] = d[1]/l[1]
88 for i in range(2, m):
89     l[i] = a_1[i] - c[i]*u1[i-1]
90     u1[i] = b_1[i]/l[i]
91     z[i] = (d[i] - c[i]*z[i-1])/l[i]
92 l[m] = a_1[m] - c[m]*u1[m-1]
93 z[m] = (d[m] - c[m]*z[m-1])/l[m]
94
95 w_1[0] = alph
96 w_1[m+1] = beta
97 w_1[m] = z[m]
98 for i in range(m-1, 0, -1):
99     w_1[i] = z[i] - u1[i]*w_1[i+1]
100
101 # =====
102 # Comparison table (Part b)
103 # =====
104 print("\nComparison Table")
105 print(f"{'x':>10} {'Shooting':>15} {'FD':>15} "
106       f"{'Exact':>15} {'Err% (Shoot)':>15} {'Err% (FD)':>15}")
107 print("-"*90)
108 for i in range(m+1):
109     x_val = a + i*h1
110     W_shoot = u[1,i] + w[2,0]*v[1,i] if i <= n else float('nan')
111     W_fd = w_1[i]
112     exact = 110*(a/x_val)*((b-x_val)/(b-a))
113     e_shoot = abs((W_shoot-exact)/exact)*100 if exact else float('nan')
114     e_fd = abs((W_fd-exact)/exact)*100 if exact else float('nan')
115     print(f"{'x_val':10.4f} {'W_shoot':15.6f} {'W_fd':15.6f} "
116           f"{'exact':15.6f} {'e_shoot':15.6f} {'e_fd':15.6f}")

```

Listing 1: Problem 1 – Shooting Method and Finite Difference Method

Table 1: Comparison of Shooting, Finite Difference, and Exact solutions for Problem 1

r	Shooting	FD	Exact	Error% (Shoot)	Error% (FD)
2.0	110.000000	110.000000	110.000000	0.000000	0.000000
2.5	66.020784	66.000000	66.000000	0.031490	0.000000
3.0	36.682707	36.666667	36.666667	0.043747	0.000000
3.5	15.722068	15.714286	15.714286	0.049524	0.000000

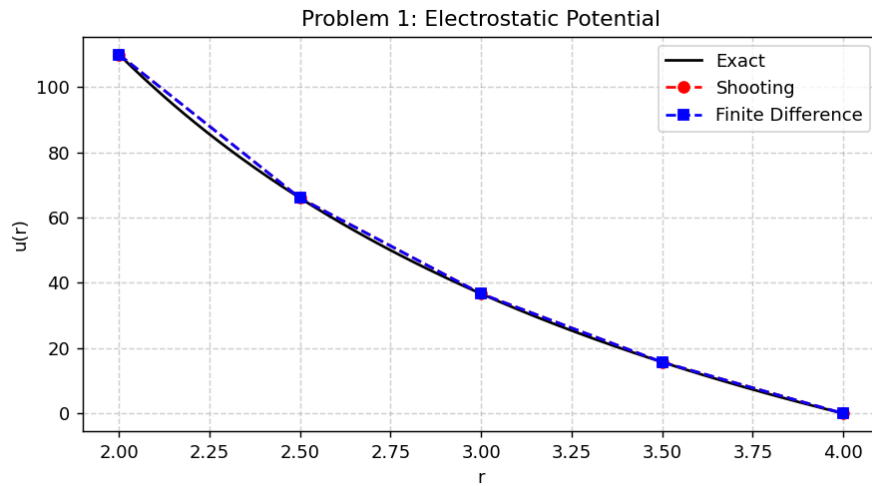


Figure 1: Electrostatic potential $u(r)$: Shooting, Finite Difference, and Exact solution.

Output.

Comments (Part c).

- **Finite Difference Method** achieves the best precision accuracy (error $\approx 0\%$)
- **Linear Shooting Method** introduces a small but non-zero error (up to $\approx 0.05\%$)

Problem 2.

Consider the nonlinear IVP governing current flow in a vacuum tube :

$$\frac{d^2x}{dt^2} - \mu(1 - x^2)\frac{dx}{dt} + x = 0, \quad x(0) = 1, \quad x'(0) = 0,$$

where $\mu > 0$ governs the degree of nonlinearity.

1. Convert the IVP to a system of first-order ODEs.
2. Plot the phase portrait for $\mu = 0, 1, 2$.
3. Plot $x(t)$ for $0 \leq t \leq 10$.
4. Interpret the solution physically.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 t_max = 10.0
5 dt = 0.005
6 n = int(t_max / dt)
7 t = np.linspace(0, t_max, n + 1)
8 mu_values = [0, 1, 2]
9
10 fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(8, 10))
11
12 for mu in mu_values:
13     x1 = np.zeros(n + 1)
14     x2 = np.zeros(n + 1)
15     x1[0] = 1.0
16     x2[0] = 0.0
17
18     for i in range(n):
19         cx1 = x1[i]
20         cx2 = x2[i]
21
22
23         dx1_1 = cx2
24         dx2_1 = mu*(1 - cx1**2)*cx2 - cx1
25
26         xh1 = cx1 + 0.5*dt*dx1_1
27         xh2 = cx2 + 0.5*dt*dx2_1
28         dx1_2 = xh2
29         dx2_2 = mu*(1 - xh1**2)*xh2 - xh1
30
31         xh12 = cx1 + 0.5*dt*dx1_2
32         xh22 = cx2 + 0.5*dt*dx2_2
33         dx1_3 = xh22
34         dx2_3 = mu*(1 - xh12**2)*xh22 - xh12
35
36         xe1 = cx1 + dt*dx1_3
37         xe2 = cx2 + dt*dx2_3
38         dx1_4 = xe2
39         dx2_4 = mu*(1 - xe1**2)*xe2 - xe1
40
41     x1[i+1] = cx1 + (dt/6)*(dx1_1 + 2*dx1_2 + 2*dx1_3 + dx1_4)

```

```
42     x2[i+1] = cx2 + (dt/6)*(dx2_1 + 2*dx2_2 + 2*dx2_3 + dx2_4)
43
44     ax1.plot(x1, x2, label=f'mu = {mu}')
45     ax2.plot(t, x1, label=f'mu = {mu}')
46
47 ax1.set_title('Phase Portrait: Velocity vs Position')
48 ax1.set_xlabel('Position (x)')
49 ax1.set_ylabel("Velocity (dx/dt)")
50 ax1.grid(True, linestyle='--', alpha=0.7)
51 ax1.legend()
52
53 ax2.set_title('Solution over Time: Position vs Time')
54 ax2.set_xlabel('Time (t)')
55 ax2.set_ylabel('Position (x)')
56 ax2.grid(True, linestyle='--', alpha=0.7)
57 ax2.legend()
58
59 plt.tight_layout()
60 plt.show()
```

Listing 2: Problem 2

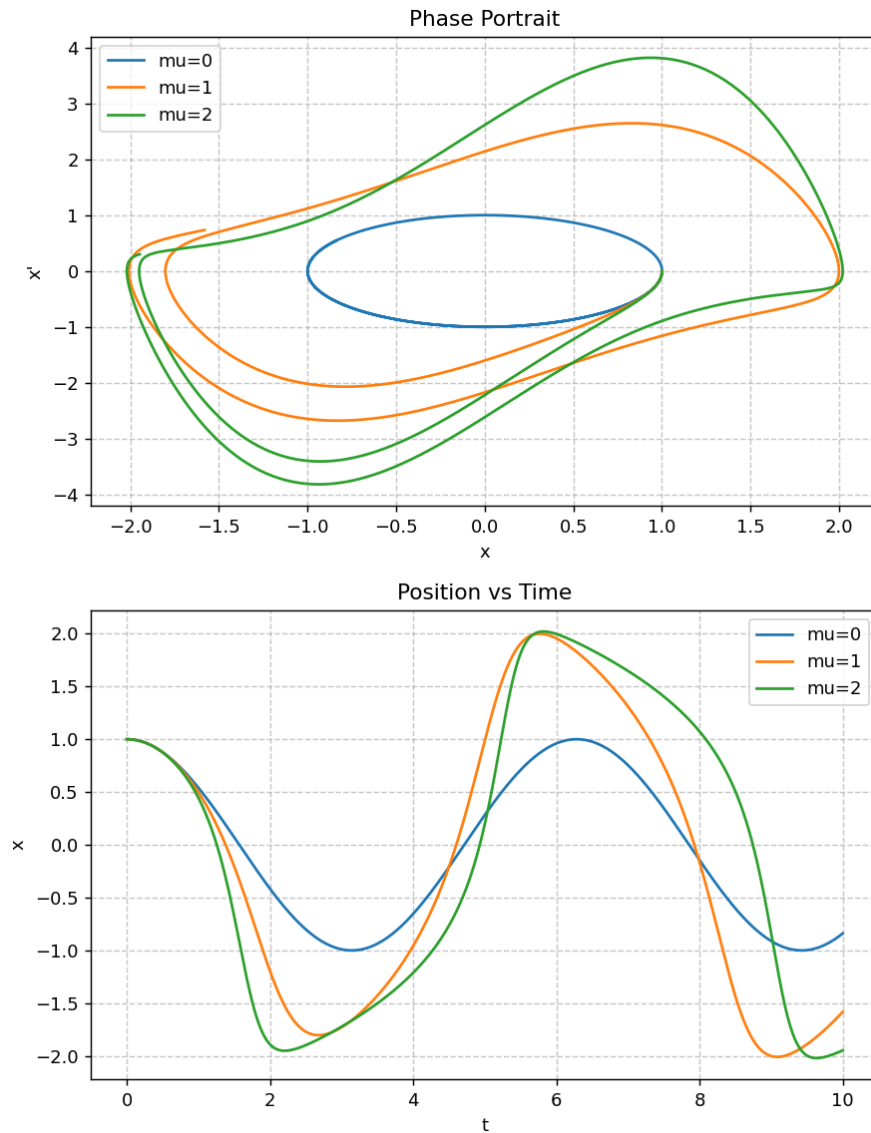


Figure 2: Phase portrait (top) and time-domain solution (bottom).

Output.

Physical Interpretation (Part d).

- $\mu = 0$: Behaves like a simple harmonic oscillator.
- $\mu = 1$: There is a moderate non linearity
- $\mu = 2$: Very strong non linearity

Problem 3.

Two species x and y compete for the same food supply according to

$$\frac{dx}{dt} = x(k_1 - k_2x - k_3y), \quad \frac{dy}{dt} = y(k_4 - k_5x - k_6y),$$

with $k_1 = 3.0$, $k_2 = 0$, $k_3 = 0.002$, $k_4 = -0.5$, $k_5 = -0.0006$, $k_6 = 0$ and initial populations $x(0) = 1000$, $y(0) = 500$ for $0 \leq t \leq 7$.

1. Approximate the population of each species.
2. Plot the time-evolution and phase portrait.
3. Determine whether a stable equilibrium exists.

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  t_max = 7.0
5  dt = 0.001
6  n = int(t_max / dt)
7  t = np.linspace(0, t_max, n + 1)
8  x = np.zeros(n + 1)
9  y = np.zeros(n + 1)
10 x[0] = 1000.0
11 y[0] = 500.0
12
13 k1=3.0; k2=0.0; k3=0.002; k4=-0.5; k5=-0.0006; k6=0.0
14
15 for i in range(n):
16     cx = x[i]
17     cy = y[i]
18
19     dx1 = cx*(k1 - k2*cx - k3*cy)
20     dy1 = cy*(k4 - k5*cx - k6*cy)
21     xh = cx + 0.5*dt*dx1
22     yh = cy + 0.5*dt*dy1
23     dx2 = xh*(k1 - k2*xh - k3*yh)
24     dy2 = yh*(k4 - k5*xh - k6*yh)
25     xh2 = cx + 0.5*dt*dx2
26     yh2 = cy + 0.5*dt*dy2
27     dx3 = xh2*(k1-k2*xh2-k3*yh2)
28     dy3 = yh2*(k4-k5*xh2-k6*yh2)
29     xe = cx + dt*dx3
30     ye = cy + dt*dy3
31     dx4 = xe*(k1 - k2*xe - k3*ye)
32     dy4 = ye*(k4 - k5*xe - k6*ye)
33
34     x[i+1] = cx + (dt/6)*(dx1 + 2*dx2 + 2*dx3 + dx4)
35     y[i+1] = cy + (dt/6)*(dy1 + 2*dy2 + 2*dy3 + dy4)
36
37 fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(8, 10))
38
39 ax1.plot(t, x, label='Species x')
40 ax1.plot(t, y, label='Species y')
41 ax1.set_title('Population over Time')

```



```

42 ax1.set_xlabel('Time')
43 ax1.set_ylabel('Population')
44 ax1.grid(True)
45 ax1.legend()
46
47 ax2.plot(x, y, color='purple')
48 ax2.set_title('Phase Portrait')
49 ax2.set_xlabel('Species x')
50 ax2.set_ylabel('Species y')
51 ax2.grid(True)
52
53 plt.tight_layout()
54 plt.show()

```

Listing 3: Problem 3

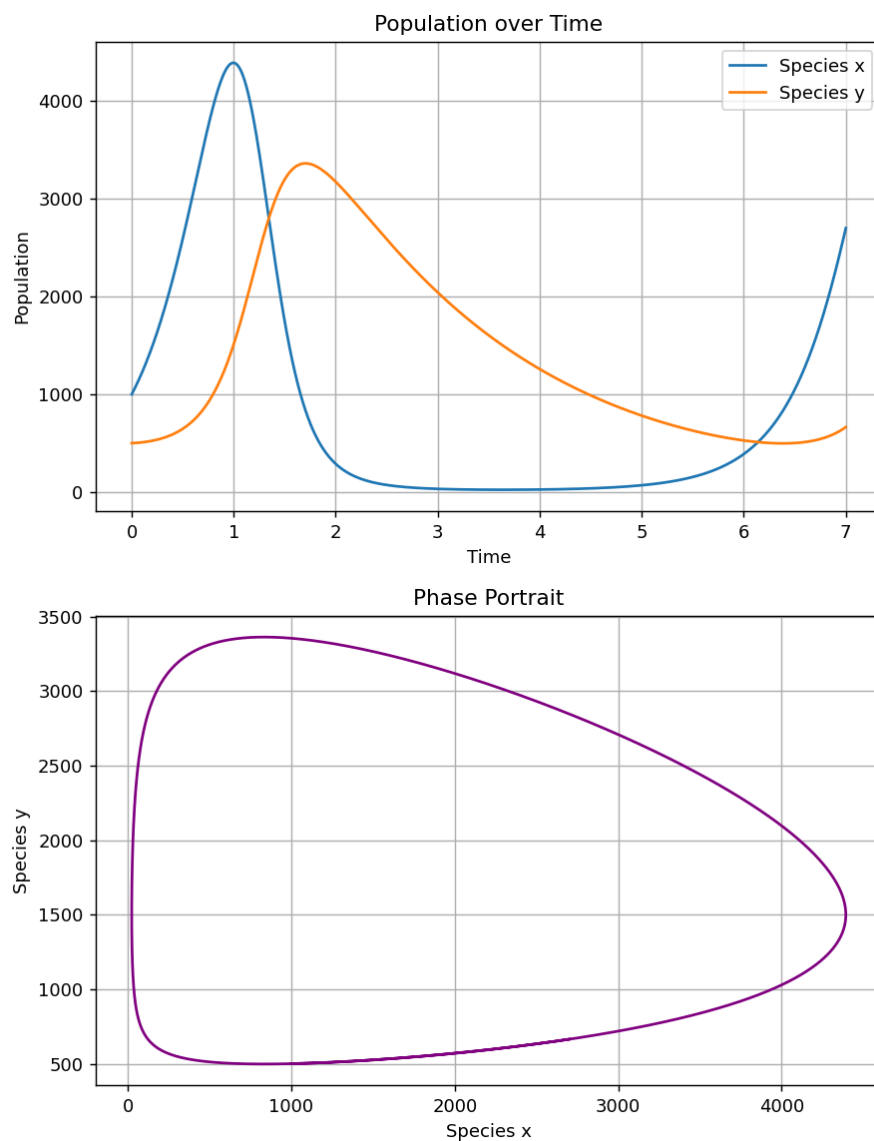


Figure 3: Competing species – population vs. time (top) and phase portrait (bottom).

Output.

Stability Analysis (Part c).

- The phase portrait shows a trajectory spiraling into this fixed point, confirming it is a **stable equilibrium** (stable node or stable spiral).
- Ecologically, both species coexist at these population levels; neither drives the other to extinction under the given parameters.

Problem 4.

Let $P(t)$ be the population at year t . With a constant birth rate b and a density-dependent death rate $d = kP(t)$, the growth is modeled by the logistic ODE

$$\frac{dP}{dt} = (b - kP)P, \quad P(0) = 50\,976,$$

with $b = 2.9 \times 10^{-2}$ and $k = 1.4 \times 10^{-7}$.

1. Approximate the population for years 1–5 .
2. Compare with the exact (analytical) solution.

```

1 import numpy as np
2
3 t_max = 5.0
4 dt = 1.0
5 n = int(t_max / dt)
6 t = np.linspace(0, t_max, n + 1)
7 P = np.zeros(n + 1)
8 P_exact = np.zeros(n + 1)
9 P[0] = 50976.0
10 b = 2.9e-2
11 k = 1.4e-7
12
13 for i in range(n):
14     cp = P[i]
15     dP1 = cp*(b - k*cp)
16     Ph = cp + 0.5*dt*dP1
17     dP2 = Ph*(b - k*Ph)
18     Ph2 = cp + 0.5*dt*dP2
19     dP3 = Ph2*(b - k*Ph2)
20     Pe = cp + dt*dP3
21     dP4 = Pe*(b - k*Pe)
22     P[i+1] = cp + (dt/6)*(dP1 + 2*dP2 + 2*dP3 + dP4)
23
24 A = P[0] / (b - k*P[0])
25
26 print("Year | Approx Population | Exact Population | Absolute Error")
27 print("-" * 65)
28 for i in range(n + 1):
29     P_exact[i] = (A*b*np.exp(b*t[i])) / (1 + k*A*np.exp(b*t[i]))
30     error = abs(P_exact[i] - P[i])
31     print(f"{t[i]:4.0f} | {P[i]:17.5f} | {P_exact[i]:16.5f} | {error}
      :.5e}")

```

Listing 4: Problem 4

Table 2: Logistic population growth – RK4 vs. exact solution

Year	Approx Population	Exact Population	Absolute Error
0	50 976.00000	50 976.00000	7.28e-12
1	52 098.69476	52 098.69476	1.59e-06
2	53 237.68651	53 237.68651	3.17e-06
3	54 392.84335	54 392.84336	4.73e-06
4	55 564.01586	55 564.01587	6.28e-06
5	56 751.03676	56 751.03677	7.82e-06

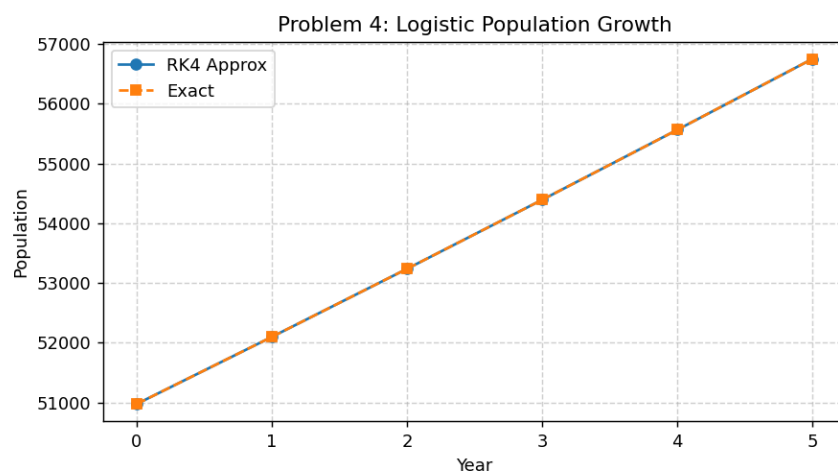


Figure 4: Logistic population growth: RK4 approximation vs. exact solution.